

Imperial College London

MSC INDIVIDUAL PROJECT

IMPERIAL COLLEGE LONDON

DEPARTMENT OF COMPUTING

Agents to Assist Data Visualisation

Author:
Jingyi Mao

Supervisor:
Prof. Peter McBrien

Second Marker:

June 2, 2026

Contents

1	Introduction	1
1.1	Project Motivation	1
1.2	Project Objectives	1
2	Literature Review	3
2.1	Data Visualisation	3
2.1.1	The Grammar of Graphics	3
2.1.2	Visualisation Specification Languages	5
2.2	Conceptual Modelling of Relational Data	6
2.2.1	ER Schemas	6
2.2.2	Visualisation Schema Patterns	7
2.3	LLMs for Schema Reasoning and Visualisation	8
2.3.1	Prompt Engineering and Structured Output	8
2.3.2	LLMs over Structured and Tabular Data	9
2.3.3	LLM-based Visualisation Systems	10
2.4	Related Work	11
2.4.1	Automated Visualisation Recommendation	11
2.4.2	Research Gap	11
3	Technical Achievements	12
3.1	Mondial Schema Representation	12
3.2	Schema Pattern Identification Framework	12
3.3	Experimental Evaluation	13
4	Project Plan	14
4.1	Natural Language Query Interpretation	15
4.2	Visualisation Recommendation	15
4.3	Data Retrieval and Visualisation Generation	15
4.4	Evaluation	15
4.5	Timeline	16
4.6	Definition of Success	16

Chapter 1

Introduction

1.1 Project Motivation

Data visualisation is an important technique for helping users understand complex datasets and discover meaningful patterns. However, many current data visualisation tools operate at a relatively low level of abstraction. In particular, users are often required to select tables, columns, transformations, and visual encodings directly from tabular data representations. This can be difficult for users who understand the real-world concepts in a domain, but are less familiar with the underlying database structure [1].

A key limitation of this table-centred approach is that it often ignores the conceptual model of the data. Conceptual models, such as Entity-Relationship (ER) schemas, describe data in terms of entities, relationships, attributes, and cardinality constraints [1]. These structures can provide important guidance for visualisation selection. For example, a many-many relationship represents an association where one instance of an entity can be related to multiple instances of another entity, and vice versa. In the Mondial database, the relationship between countries and continents is an example: one continent contains many countries, while some countries may be associated with more than one continent. Such structures are often more suitable for network-based visualisations [1], because the visualisation needs to show links between two sets of related entities. Therefore, reasoning over conceptual schema structures can help bridge the gap between how users understand data and how visualisation tools represent it internally.

Previous work by McBrien and Poulouvasilis proposed the use of visualisation schema patterns to connect conceptual data structures with suitable visualisations. This approach has also been implemented and explored in earlier systems such as Bull’s schema-driven visualisation tool [2] and Hannan’s VizER system [3]. However, existing approaches mainly rely on manually designed rules or explicit pattern-matching procedures. With recent advances in Large Language Models (LLMs), there is an opportunity to investigate whether LLMs can perform this type of schema-level reasoning automatically.

This project explores the use of LLMs to reason over relational schemas, identify conceptual visualisation schema patterns, and select appropriate visualisations. Rather than requiring users to manually inspect tables and columns, the long-term goal is to support a more intuitive workflow in which a user can express a visualisation request in natural language, and the system can retrieve the relevant data, identify the underlying schema pattern, recommend a suitable visualisation, and generate an executable visualisation specification.

The project uses the Mondial database [4] as its primary experimental environment. Mondial is a large geographical database containing entities such as countries, cities, provinces, rivers, lakes, and religions, together with rich relationship and cardinality information. Its well-documented schema and diverse conceptual structures make it a suitable benchmark for evaluating schema-level reasoning, visualisation schema pattern identification, and automated visualisation recommendation.

1.2 Project Objectives

The main objective of this project is to investigate whether Large Language Models can be used to support conceptual-model-based data visualisation. Using the Mondial database as a case study,

the project focuses on whether LLMs can reason about relational schema information, including primary keys, foreign keys, and cardinality-related structures, and use this reasoning to identify appropriate visualisation schema patterns.

The project has four main objectives:

- To represent the Mondial database schema in a machine-readable format that can be provided to an LLM as context.
- To evaluate whether LLMs can reason over selected tables and attributes and correctly classify them into visualisation schema patterns, including Basic Entity, Weak Entity, One-Many Relationship, Many-Many Relationship, and Reflexive Relationship.
- To investigate whether LLMs can recommend suitable visualisations based on the identified schema patterns and generate structured outputs that can be used by a visualisation library.
- To extend the current schema pattern identification approach towards an end-to-end natural-language-to-visualisation pipeline, including natural language query interpretation, data retrieval, visualisation recommendation, and visualisation specification generation.

The initial stage of the project focuses on prompt-based schema reasoning over the Mondial database. Later stages will investigate visualisation generation and interaction. The project will also consider whether prompt engineering is sufficient for this task or whether training or fine-tuning a custom model is required.

The expected final output is a prototype system that demonstrates how LLM-based agents can assist data visualisation by using conceptual schema information rather than relying only on low-level tabular data. The success of the project will be evaluated through the accuracy of schema pattern identification, the appropriateness of visualisation recommendations, and the ability of the system to generate executable visualisations from user requests.

Chapter 2

Literature Review

2.1 Data Visualisation

2.1.1 The Grammar of Graphics

Traditional visualisation systems are commonly organised around predefined chart types such as bar charts, pie charts, and line charts. In these systems, visualisation design is typically treated as the selection of a fixed chart template. However, this chart-centred approach limits flexibility and makes it difficult to represent more complex relationships within data. From a practical perspective, chart-based systems can only provide a limited number of predefined visualisations, making it difficult to support the diversity of real-world data analysis tasks [5]. In addition, treating each chart as an independent object often leads to unnecessarily complex system design, since similar graphical operations cannot be efficiently reused across different chart types [5]. Extending such systems with new visualisations therefore requires additional specialised code, reducing both flexibility and scalability.

To address this limitation, Wilkinson introduced *The Grammar of Graphics* [5], which provides a formal and systematic framework for constructing visualisations through compositional rules rather than predefined chart categories. The concept of “grammar” was influenced by Chomsky’s transformational grammar theory [6], where grammar is defined as a formal system of rules capable of generating valid expressions in a language. Chomsky [7] argued that diverse natural language expressions can be explained through an underlying deep grammatical structure, and Wilkinson extended this idea to statistical graphics. Under this framework, visualisations are interpreted as combinations of reusable graphical components and transformations instead of isolated chart types. This perspective enables diverse visualisations to be generated systematically from reusable primitives, while also revealing structural relationships between different visual forms.

Wilkinson further organises visualisation generation into a sequence of interconnected and ordered components, beginning with the data source. The diagram 2.1 illustrates how a chart is constructed within Wilkinson’s framework. Each stage of the process is explained below:

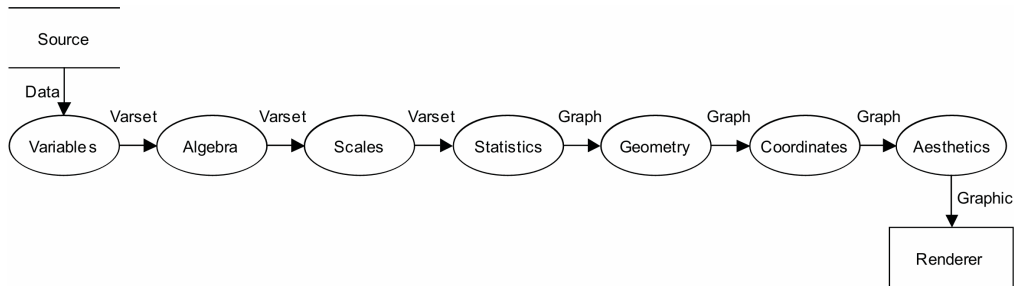


Figure 2.1: Data-flow diagram of chart construction in Wilkinson’s framework. Source: [5].

- **Variables:** Variables are the data attributes selected from the original data source and a set of variables are called a **Varset** [5]. They determine what information will be represented, such as numerical measurements, categories, or labels. For example, variables such as population, income, or country name may be extracted from a dataset for further visual encoding. Transformations are included in this step to make meaningful statistical operations and enable the creation of new variables, aggregates, and other forms of summaries.
- **Algebra:** Algebra defines the operations that allow us to combine variables and specify dimensions of graphs [5]. Wilkinson proposed three basic operators that are necessary and sufficient for statistical graphics: cross, blend, and nest, which can be implemented in SQL through operations such as `INNER JOIN`, nested queries, and `UNION ALL`, respectively. Based on these fundamental operators, more complex operations can then be constructed, such as exponentiation and dot-cross. This determines the logical structure of the graphic before visual representation takes place.
- **Scales:** Scales maps a set of variables into visual measurement spaces [5]. It determines how numerical or categorical variables are translated into positions, sizes, colours, shapes, or other visual properties within a graphic. Wilkinson also introduces scale transformations, such as logarithmic and power transformations, which modify the representation of data while preserving important relationships. These transformations are not merely aesthetic adjustments, but analytical tools that help reveal patterns, compress extreme values, or improve interpretability.
- **Statistics:** Statistics are responsible for transforming raw data into more meaningful representational forms before graphical rendering [5]. These statistical methods, including bin, summary, region, smooth, and link, help reveal patterns, trends, and structures that may not be immediately visible in the raw dataset [5]. By integrating statistical computation into the visualisation grammar, graphics can represent both raw observations and derived summaries in a consistent and systematic manner.
- **Geometry:** Geometry defines the geometric objects used to represent data in a visualisation [5]. Wilkinson groups these objects into three categories: functions, partitions, and networks. Functions include points, lines, areas, intervals, and paths. Partitions divide data into regions using polygons and contours, and networks represent relationships through connected edges. By building graphics from a small set of geometric primitives rather than predefined chart types, it provides a flexible framework for creating diverse visualisations. Wilkinson also introduces collision modifiers, such as stacking, dodging, and jittering, to reduce overlap between graphical elements.
- **Coordinates:** Coordinates define the spatial framework in which geometric objects are displayed [5]. Wilkinson separates coordinate systems from geometric objects, allowing the same geometry to be represented in different ways through transformations of the underlying space. By treating coordinates as an independent component of the visualisation grammar, the same data and geometry can produce different visual representations without altering their underlying structure.
- **Aesthetics:** Aesthetics define the visual attributes used to perceive and distinguish geometric objects [5]. Wilkinson describes how data can be mapped to visual channels such as size, shape, colour, texture, and transparency, enabling graphical elements to convey additional information beyond their geometric position. By separating aesthetics from geometry, the Grammar of Graphics allows the same geometric structure to be represented through different visual encodings.

Another important aspect of the Grammar of Graphics is its declarative specification paradigm. Unlike imperative libraries such as Matplotlib, where users explicitly define drawing procedures step by step, declarative approaches describe the relationships between data and visual representations while leaving rendering details to the underlying system [8]. Wilkinson formalised this idea through six specification statements: `DATA`, `TRANS`, `SCALE`, `COORD`, `ELEMENT`, and `GUIDE`. This framework has had a significant influence on modern visualisation systems, including ggplot2 [9, 10] and Vega-Lite [11], which enable users to specify what should be visualised rather than how it should be rendered.

The Grammar of Graphics has become one of the theoretical foundations of modern visualisation specification languages and automatic visualisation systems. Many widely used visualisation frameworks are built upon its compositional and declarative design principles [8]. By representing visualisations as structured combinations of reusable components, the framework enables computational reasoning over visualisation design and supports flexible visualisation generation beyond predefined chart templates.

This perspective is particularly important for this schema-based and LLM-driven visualisation project, where the objective is not only to select chart types, but also to reason over data structures, schema patterns, and visualisation composition. In this project, the Grammar of Graphics forms an important conceptual foundation for constructing visualisations and for exploring whether Large Language Models can perform schema-level reasoning to automatically generate appropriate visualisations.

2.1.2 Visualisation Specification Languages

Visualisation specification languages provide a structured and declarative approach to visualisation design. Instead of explicitly defining rendering procedures, users specify the relationships between data and visual encodings, while the underlying system determines how the visualisation is executed. By separating specification from execution, declarative languages simplify visualisation development while enabling system-level optimisation and cross-platform deployment [8]. Most modern visualisation specification languages are influenced by Wilkinson’s Grammar of Graphics [5] or extensions of its underlying principles.

One of the earliest direct implementations of the Grammar of Graphics was GPL, used in IBM SPSS Statistics [12]. GPL introduced a formal grammar-based framework for specifying graphics through structured statements defining data sources, scales, guides, and graphical elements. However, GPL is considered relatively low-level and requires detailed specification from the user, resulting in a steep learning curve [3].

To simplify the construction of statistical graphics, Wickham introduced ggplot2 and the concept of the layered grammar of graphics, an extension and reorganisation of Wilkinson’s Grammar of Graphics [9, 10]. Rather than introducing new graphical components, the layered grammar reorganises Wilkinson’s framework around the concept of layers, where data, mappings, statistical transformations, and geometric objects are combined into self-contained visual units [13]. This layer-based structure simplifies the construction and modification of graphics while retaining the flexibility of grammar-based visualisation design. Nevertheless, ggplot2 is primarily designed for static statistical graphics and remains tightly coupled to the R programming language.

While grammar-based approaches focus on declarative visual specifications, D3 adopts a lower-level data-driven approach to visualisation design [14]. Rather than introducing a new graphical grammar, D3 enables developers to directly bind data to native web document elements and manipulate them through JavaScript transformations. By operating directly on the Document Object Model (DOM), D3 provides exceptional flexibility, compatibility with existing web standards, and support for rich interaction and animation [14]. However, this flexibility comes at the cost of increased implementation complexity, as users must explicitly specify visual structures and rendering behaviour. The expressive power of D3 made it highly influential for web-based visualisation, but its relatively low level of abstraction motivated the development of higher-level declarative grammars such as Vega [15] and Vega-Lite [11].

Vega and Vega-Lite represent two influential declarative visualisation grammars that occupy different positions in the design space of visualisation specification languages. Vega is a low-level visualisation grammar that represents visualisations through structured JSON specifications, providing fine-grained control over data transformations, scales, graphical marks, and interactive behaviours [15]. This flexibility makes Vega suitable for highly customised visualisations and visual analytics systems, but often results in relatively verbose specifications. Vega-Lite was introduced as a higher-level grammar of interactive graphics that prioritises conciseness and rapid authoring [11]. It represents visualisations through compact encoding specifications and view composition operators while relying on rule-based defaults to infer unspecified details. Vega-Lite specifications are automatically compiled into lower-level Vega specifications, sacrificing some expressive power in exchange for substantially simpler and more readable specifications. Consequently, Vega and Vega-Lite illustrate the broader trade-off between expressive flexibility and specification simplicity in visualisation language design.

A different approach to visualisation specification was introduced through Tableau’s VizQL [16], which integrates data querying and visual specification within a single framework. Rather than requiring users to explicitly define queries or graphical properties, VizQL translates interactive actions, such as dragging fields onto rows, columns, or encoding shelves, into visual queries that simultaneously determine both the underlying database operations and the resulting visual representation. In this way, VizQL serves as a bridge between relational data and visualisation, describing not only how data should be displayed but also how the required data should be retrieved and aggregated. This abstraction significantly lowers the barrier to creating visualisations and laid the foundation for automatic visualisation recommendation systems.

Together, these languages illustrate the diversity of approaches to visualisation specification. GPL and ggplot2 extend the principles of the Grammar of Graphics through declarative grammar-based design, D3 emphasises direct manipulation and flexibility for web-based visualisation, Vega and Vega-Lite explore different trade-offs between expressive power and specification simplicity, while VizQL integrates visualisation specification with database querying. Despite their differences, all of these approaches aim to represent visualisations in a structured form that separates design intent from implementation details. This structured representation provides an important foundation for automated visualisation systems, enabling visualisations to be generated, analysed, and recommended programmatically.

2.2 Conceptual Modelling of Relational Data

2.2.1 ER Schemas

The Entity-Relationship (ER) schema is a conceptual modelling approach used to represent the semantics of an application domain independently of its implementation in a relational database. Rather than describing data in terms of tables and columns, the ER schema captures the conceptual structure of an application domain through entities, relationships, attributes, and cardinality constraints. Entities correspond to classes of real-world objects or concepts, relationships describe associations between entities, attributes provide descriptive information about entities or relationships, and cardinality constraints specify participation restrictions between related entities. These core components can be defined as follows:

- **Entity:** An entity E represents a set of objects that conceptually belong to the same type of thing. In ER schemas, entities are typically identified from nouns that describe the objects within the application domain [17].
- **Relationship:** A relationship R represents a set of tuples of objects, where each tuple corresponds to a conceptual association between entities E_1, E_2 . Relationships are commonly identified from verbs in the application domain [17].
- **Attribute:** Attributes describe the properties of entities or relationships [18]. Attributes may be classified as mandatory, optional, or key attributes. Mandatory attributes are defined for every entity instance, optional attributes may be absent for some instances, and key attributes uniquely identify entities within an entity set [17].
- **Cardinality Constraints:** Cardinality constraints specify the minimum and maximum number of times an entity may participate in a relationship. An upper bound constraint defines the maximum number of relationship instances in which an entity can appear, while a lower bound constraint defines the minimum number of required participations [17].

In addition to entities, relationships, attributes, and cardinality constraints, ER schemas may also represent subset or ISA (is-a) hierarchies. A subset constraint indicates that the instances of one entity form a subset of another entity. For example, a *manager* entity may be modelled as a specialised subset of the *person* entity. Such hierarchies allow ER schemas to capture inheritance and specialisation relationships within a domain [17].

The following Figure 2.2 illustrates a simple ER schema involving three entities denoted by rectangles: *person*, *department*, and *manager*. The *person* entity has three attributes: *salary_number*, *name*, and *bonus*. The underlined attribute *salary_number* represents the key attribute used to identify each person. The question mark after *bonus* indicates that this is an optional attribute,

meaning that a bonus value may not be recorded for every person. Similarly, the *department* entity is identified by the key attribute *dname*, while the *manager* entity has an additional attribute *mobile number*.

The diagram also contains a relationship, *works in*, represented by a diamond between *person* and *department*. The cardinality constraint near *person* is 1 : 1, indicating that each person must participate in exactly one *works in* relationship. In other words, every person works in one department. The cardinality constraint near *department* is 0 : *N*, indicating that a department may have zero, one, or many persons associated with it.

The diagram further includes a hierarchy, represented by the arrow from *manager* to *person*. This indicates that every manager is also a person, but not every person is necessarily a manager. The *manager* entity therefore specialises the *person* entity by adding information that is specific to managers, such as the *mobile number* attribute.

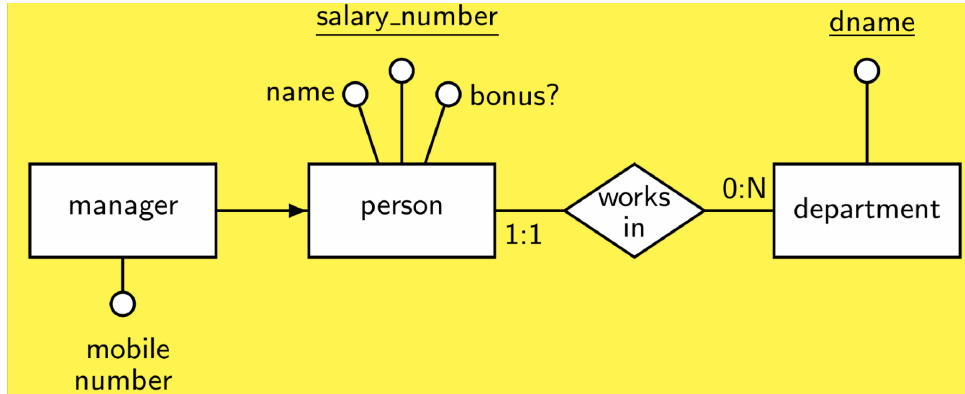


Figure 2.2: Example Entity-Relationship Schema. Source: [17].

The ability of ER schemas to explicitly represent entities, relationships, attributes, and cardinalities makes them particularly suitable for schema-level reasoning. The concepts introduced above form the basis for the Mondial database, which is used throughout this project as the primary evaluation dataset. Mondial is a large geographical database containing entities such as countries, cities, provinces, rivers, lakes, languages, and religions, together with rich relationship and cardinality information. Its conceptual ER schema provides a realistic benchmark for evaluating schema-level reasoning and visualisation pattern matching.

2.2.2 Visualisation Schema Patterns

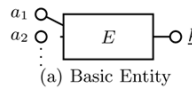
McBrien and Poulouvasilis [1] proposed a set of visualisation schema patterns that establish a conceptual mapping between ER schema and visualisations. A visualisation schema pattern describes a recurring conceptual structure within an ER schema and the classes of visualisations that are suitable for representing that structure.

The approach is based on the observation that different ER structures naturally support different forms of visual representation. By matching subgraphs of an ER schema against predefined visualisation patterns, it becomes possible to identify visualisations that are structurally compatible with the underlying data model. This allows visualisation recommendations to be guided by conceptual schema information rather than solely by individual data attributes.

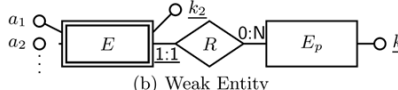
Based on an analysis of common visualisation techniques, five visualisation schema patterns were identified: Basic Entity, Weak Entity, One-Many Relationship, Many-Many Relationship, and Reflexive Relationship [1]. Each pattern corresponds to a particular ER structure and is associated with one or more visualisation types. Figure 2.3 summarises the five visualisation schema patterns and their associated visualisations.

These visualisation schema patterns form the conceptual foundation of the schema-driven visualisation recommendation approach adopted in this project. The identification of schema patterns from ER schemas is therefore a key step in the subsequent visualisation selection process.

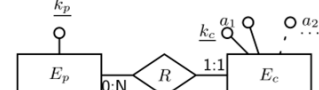
ER Schema Patterns



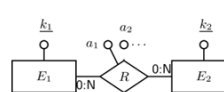
(a) Basic Entity



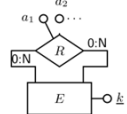
(b) Weak Entity



(c) One-Many Relationship (Hierarchy)



(d) Many-Many Relationship



(e) Reflexive

Recommended Visualisations

Basic Entity Visualisations				
Name	$ k $		mandatory	optional
Bar Chart	1..100	a_1 scalar	-	
Calendar	1..*	a_1 temporal scalar	a_2 colour	
Scatter Diagrams	1..*	a_1, a_2 scalar	a_3 colour	
Bubble Charts	1..*	a_1, a_2, a_3 scalar	a_4 colour	
Choropleth Maps	1..*	k geographical, a_1 colour	-	
Word Clouds	1..*	k lexical, a_1 scalar	a_2 colour	

Weak Entity Visualisations					
Name	$ k_1 $	$ k_2 $	complete	mandatory	optional
Line Chart	1..20	1..*	no	k_2, a_1 scalar	a_2 scalar
Stacked Bar Chart	1..20	1..20	yes	a_1 scalar	-
Grouped Bar Chart	1..20	1..20	no	a_1 scalar	-
Spider Chart	3..10	1..20	yes	a_1 scalar	-

One-many relationships					
Name	$ k_1 $	$ k_2 $	per k_1	mandatory	optional
Tree Map	1..100	1..100	a_1 scalar	a_2 colour	
Hierarchy Tree	1..100	1..100	-	a_1 colour	
Circle Packing	1..100	1..100	a_1 scalar	a_2 colour	

Many-many relationships					
Name	$ k_1 $	$ k_2 $	reflexive	mandatory	optional
Sankey	1..20	1..20	no	a_1 scalar	a_2 colour
Chord	1..100	1..100	yes	a_1 scalar	a_2 colour

Figure 2.3: ER Schema Patterns and Recommended Visualisations Adapted from McBrien and Poulouvassilis [1]

2.3 LLMs for Schema Reasoning and Visualisation

Large Language Models (LLMs) have demonstrated strong capabilities in natural language understanding [19], reasoning [20], code generation [21], and structured prediction [22]. Recent research suggests that these capabilities can be extended beyond traditional language tasks to support reasoning over structured data [23], database schemas [24, 25], and visualisation workflows [26]. This section reviews the key concepts underlying the use of LLMs for schema reasoning and visualisation generation.

2.3.1 Prompt Engineering and Structured Output

Large Language Models (LLMs) can often be adapted to new tasks without additional model training through prompt engineering [19]. More broadly, prompting has emerged as a new paradigm for natural language processing, often referred to as prompt-based learning [27]. Unlike traditional supervised learning, which directly learns a mapping from inputs to task-specific outputs, prompt-based learning reformulates downstream tasks into forms that resemble the language modelling objectives used during pre-training. By transforming an input into a carefully designed prompt, the model can leverage knowledge acquired during large-scale pre-training to perform new tasks with little or no task-specific training [27].

Prompt engineering refers to the process of designing and refining prompts to guide a model towards a desired behaviour. A prompt may contain task instructions, contextual information, examples, constraints, or reasoning guidance that help the model interpret the user’s objective and generate appropriate outputs. The quality and structure of prompts can have a significant influence on model performance, as prompts effectively define how a task is presented to the language model [27].

The effectiveness of prompt engineering is closely related to the in-context learning capability of modern LLMs, whereby task-specific behaviour can be induced directly from information contained within the prompt rather than through parameter updates [19]. Consequently, prompt engineering has become a widely adopted approach for adapting general-purpose language models to specialised tasks, including question answering, information extraction, reasoning, code generation, and data analysis.

Several prompting strategies have been proposed in the literature and are illustrated in Figure 2.4. Zero-shot prompting provides only task instructions and relies entirely on the model’s pre-trained knowledge to perform a task without demonstrations [28]. Few-shot prompting ex-

tends this approach by incorporating a small number of input-output examples within the prompt, enabling the model to infer the desired task format and behaviour through demonstration [19]. Chain-of-thought (CoT) prompting further improves reasoning performance by encouraging the model to generate intermediate reasoning steps before producing a final answer, making complex reasoning processes more explicit and interpretable [20]. These prompting techniques have been shown to substantially improve performance across a wide range of reasoning and problem-solving tasks and have become fundamental components of many contemporary LLM-based systems.

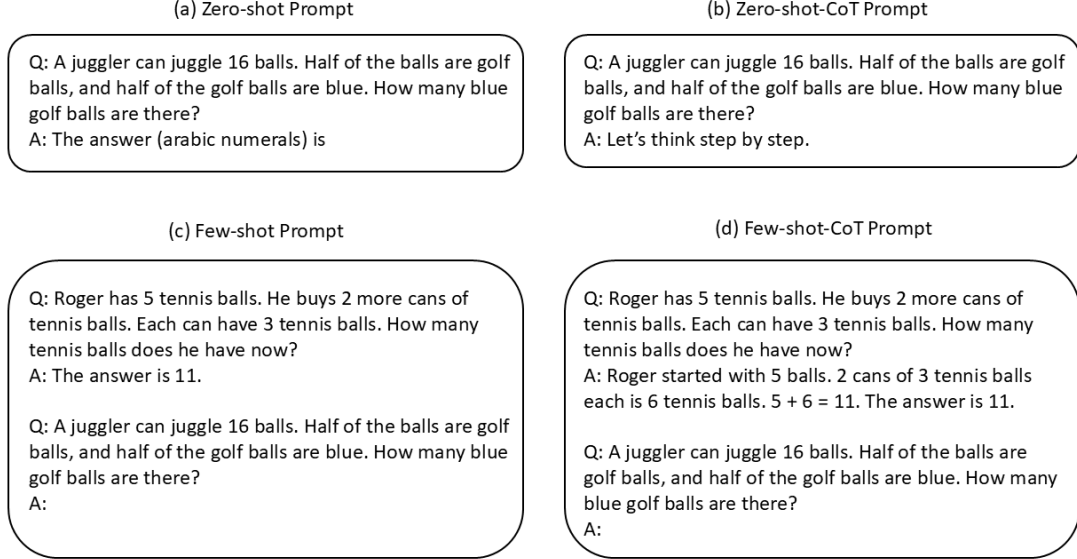


Figure 2.4: Examples of zero-shot, zero-shot-CoT, few-shot, and few-shot-CoT prompting. Adapted from Kojima et al. [28].

For many applications, prompts are designed not only to guide model behaviour but also to constrain the format of generated outputs. Structured output prompting requires LLMs to produce responses in machine-readable formats such as JSON, SQL, or XML, enabling integration with external software systems and automated processing pipelines [29]. This capability is particularly important in data-driven applications, where model outputs must often be consumed by downstream analytical or visualisation tools.

The ability of prompt engineering to elicit reasoning without additional model training is particularly relevant to this project. Different prompting strategies may significantly influence an LLM’s ability to perform complex reasoning tasks and generate structured outputs. Consequently, prompt design forms a key component of the proposed approach and will be evaluated as part of the system development and experimentation.

2.3.2 LLMs over Structured and Tabular Data

Although large language models were originally developed for natural language processing tasks, they have increasingly been applied to structured and tabular data. A representative research direction is Text-to-SQL parsing, where a natural language question is translated into an executable SQL query over a relational database [30]. This task is important because it allows non-technical users to access relational data without manually writing SQL queries.

Text-to-SQL is not only a language generation task, but also a schema reasoning task. In order to generate a correct SQL query, the model must understand both the user’s question and the structure of the underlying database schema. This includes identifying relevant tables and columns, linking natural language expressions to schema items, and reasoning over primary keys, foreign keys, column types, and join paths [30]. For example, if a question mentions provinces in a country, the model may need to infer that the province table should be joined with the country table through a foreign-key relationship.

Modern Text-to-SQL methods therefore place strong emphasis on schema linking and schema structure modelling [30]. Schema linking refers to the process of aligning words or phrases in

the natural language question with tables, columns, or values in the database. Schema structure modelling, by contrast, focuses on the internal organisation of the database itself, such as which columns belong to which tables and how tables are connected through primary-key and foreign-key relationships. Recent approaches such as RAT-SQL [31], SADGA [32] and LGESQL [33] represent the question and database schema as graph structures, where nodes may correspond to question tokens, tables and columns, and edges encode relations such as table-column membership, foreign-key links, and question-schema alignments. This graph-based encoding allows models to reason explicitly over relational structure rather than treating the schema as a flat sequence of text.

Large benchmark datasets have played an important role in evaluating these capabilities. In particular, Spider [24] has become a standard cross-domain Text-to-SQL benchmark because it requires models to generalise to unseen databases and reason over complex schemas across different domains. Compared with earlier datasets such as WikiSQL [34], which mainly contains simpler single-table queries, Spider better reflects the challenge of understanding multi-table relational schemas and generating complex SQL queries.

These developments are relevant to this project because they show that successful language-based interaction with structured data requires more than surface-level text understanding. Models must reason about the conceptual and relational structure behind the data. While Text-to-SQL systems aim to map natural language questions to SQL queries, this project focuses on mapping user intent and relational schema information to appropriate visualisation schema patterns. Therefore, research on schema linking, schema reasoning, and graph-based schema encoding provides an important foundation for using LLMs to support ER-driven visualisation recommendation.

2.3.3 LLM-based Visualisation Systems

Recent advances in Large Language Models (LLMs) have stimulated growing interest in applying language-based reasoning to visualisation tasks. Unlike traditional visualisation systems that often rely on handcrafted rules, specialised recommendation algorithms, or task-specific machine learning models, LLM-based approaches utilise prompt engineering to guide pre-trained models towards generating visualisations, visualisation specifications, or visual analytics insights.

An important precursor to this trend is NL4DV [35], which translates natural language queries into analytic specifications consisting of relevant data attributes, analytic tasks, and Vega-Lite visualisations. Although developed before the widespread adoption of modern foundation models, NL4DV illustrates the importance of semantic interpretation in connecting user intent with visualisation generation.

Subsequent work explored the direct use of large language models for visualisation generation. Chat2VIS [36] demonstrates that GPT-3 can generate visualisations directly from free-form natural language queries. By replacing complex natural language processing pipelines with prompt-guided code generation, the system highlights the potential of LLMs as end-to-end interfaces for visualisation authoring.

Similarly, LIDA [26] extends the role of language models beyond visualisation generation to support multiple stages of the visualisation workflow. The framework combines dataset summarisation, visualisation goal exploration, visualisation generation, and infographic creation within a unified LLM-driven pipeline. This work demonstrates how prompt-guided reasoning can support not only chart generation but also higher-level visual analytics tasks.

LLM4Vis [37] further investigates the use of prompting for visualisation recommendation. Instead of training task-specific recommendation models, the system converts dataset characteristics into natural language descriptions and employs few-shot prompting to recommend visualisation types and provide textual explanations for its recommendations.

Taken together, these systems demonstrate that prompt-engineered LLMs can effectively support visualisation generation, recommendation, and interaction. However, their reasoning processes remain primarily grounded in natural language descriptions, dataset summaries, and column-level data characteristics. Relatively little attention has been given to whether LLMs can reason directly over conceptual database schemas and their structural semantics.

2.4 Related Work

2.4.1 Automated Visualisation Recommendation

Automated visualisation recommendation seeks to assist users in selecting effective visual representations by automatically recommending chart types, visual encodings, or complete visualisation specifications. Research in this area has evolved from rule-based approaches towards machine learning and knowledge-driven recommendation techniques.

One of the earliest systems was Mackinlay’s APT [38], which formalised graphical design knowledge and selected visual encodings according to perceptual effectiveness criteria. This work established the idea that visualisation design principles could be represented explicitly and applied automatically.

Later systems such as Tableau Show Me [16], Voyager [39], and Voyager 2 [40] extended this rule-based paradigm by recommending visualisations based on attribute characteristics, partial visualisation specifications, and design constraints. These systems support exploratory data analysis by helping users navigate large visualisation design spaces and identify potentially useful visual representations.

Other approaches focus on identifying informative or interesting views within a dataset. SeeDB [41], for example, recommends visualisations by identifying deviations between target and reference data distributions, automatically surfacing views that are likely to reveal meaningful patterns.

More recently, machine learning approaches have sought to reduce reliance on handcrafted rules. VizML [42] formulates visualisation recommendation as a collection of prediction tasks, using statistical properties extracted from tabular datasets to infer chart types and visual encodings. KG4Vis [43] incorporates visualisation design knowledge through a knowledge graph representation, improving both recommendation quality and interpretability.

2.4.2 Research Gap

Automated visualisation recommendation systems have developed increasingly sophisticated methods for selecting visualisations from tabular data, progressing from rule-based systems to machine learning and, more recently, LLM-based approaches. However, these systems typically reason about attributes, values, and statistical properties rather than conceptual schema structures.

Conversely, conceptual modelling-based visualisation research has demonstrated that entities, relationships, weak entities, and cardinality constraints can provide valuable guidance for visualisation selection [1]. Nevertheless, existing implementations rely primarily on manually constructed rules and explicit pattern-matching procedures to identify visualisation schema patterns [2, 3].

At the same time, recent advances in large language models have demonstrated strong capabilities in structured data understanding, schema reasoning, and prompt-guided decision making. Yet relatively little research has investigated whether these capabilities can be applied to conceptual schema analysis and visualisation schema pattern identification.

Therefore, this project investigates whether prompt-engineered LLMs can reason directly over conceptual schemas, identify visualisation schema patterns, and recommend appropriate visualisations based on conceptual relationships. By combining conceptual modelling with modern language models, the project aims to bridge the gap between schema-level semantics and automated visualisation design.

Chapter 3

Technical Achievements

Several key components of the proposed system have been developed and evaluated during the first stage of the project. The work completed so far focuses on enabling LLMs to identify schema patterns from relational schema information.

3.1 Mondial Schema Representation

The experiments conducted in this project are based on the Mondial database. In order to support automated schema reasoning, structural information was extracted from the database and transformed into a machine-readable representation. For each table, the extracted metadata includes:

- table names;
- column names;
- data types;
- primary keys;
- foreign-key relationships.

The extracted schema information was represented in JSON format and provided as context to the LLM. This representation preserves the structural relationships between tables while remaining independent of any particular database implementation. By explicitly providing key constraints and relationships, the model is able to reason about conceptual structures that are not directly visible from the selected tables and attributes alone.

3.2 Schema Pattern Identification Framework

The primary technical objective completed during this stage was the development of a schema pattern identification framework.

Given a set of selected tables and attributes, the system aims to determine which schema pattern best describes the selected data. The classification task is based on the five schema patterns identified in previous work:

- Basic Entity;
- Weak Entity;
- One-Many Relationship;
- Many-Many Relationship;
- Reflexive Relationship.

The framework uses prompt engineering techniques to guide the reasoning process of the LLM. For each test case, the model is provided with:

- the Mondial schema information (served as context information);
- definitions about five schema patterns (served as context information)
- selected table names;
- selected attribute names.

The model is then required to classify the input into one of the five schema patterns. Unlike conventional Text-to-SQL systems, which use schema information to generate executable database queries, the proposed framework uses relational schema information to infer higher-level conceptual modelling structures. The objective is not to generate SQL queries, but rather to bridge the gap between relational database representations and conceptual visualisation schema patterns.

3.3 Experimental Evaluation

An initial evaluation was conducted using nine test cases derived from previous student projects based on the Mondial database. Each test case consists of a set of selected tables and attributes together with a ground-truth visualisation schema pattern. The objective was to determine whether the LLM could correctly infer the conceptual schema pattern from relational schema information alone.

Chosen Table	Chosen Columns	Groundtruth	Identified Pattern	PASS/FAIL?
Country	code, population	Basic	Basic	PASS
Lake	name, depth, elevation, dam_height	Basic	Basic	PASS
Organisation	abbreviation, established	Basic	Basic	PASS
Economy	country, gdp, unemployment	Basic with inherited key	Basic with inherited key	PASS
Country Population	country, year, population	Weak	Weak	PASS
Ethnic Group	country, name, percentage	Weak	Weak	PASS
Airport	iata_code, island, elevation	One-Many	One-Many	PASS
Encompasses	country, continent, percentage	Many-Many	Many-Many	PASS
Borders	country1, country2, length	Reflexive Many-Many	Reflexive Many-Many	PASS

Table 3.1: Test Cases for Schema Pattern Identification

Table 3.3 summarises the evaluation results. The experiments demonstrate that the proposed approach correctly identified the schema pattern for all test cases. These results provide initial evidence that LLMs can successfully reason over relational schema structures and infer schema patterns.

Chapter 4

Project Plan

The remaining stages of the project will focus on extending the current schema pattern identification prototype into a complete natural-language-to-visualisation system.

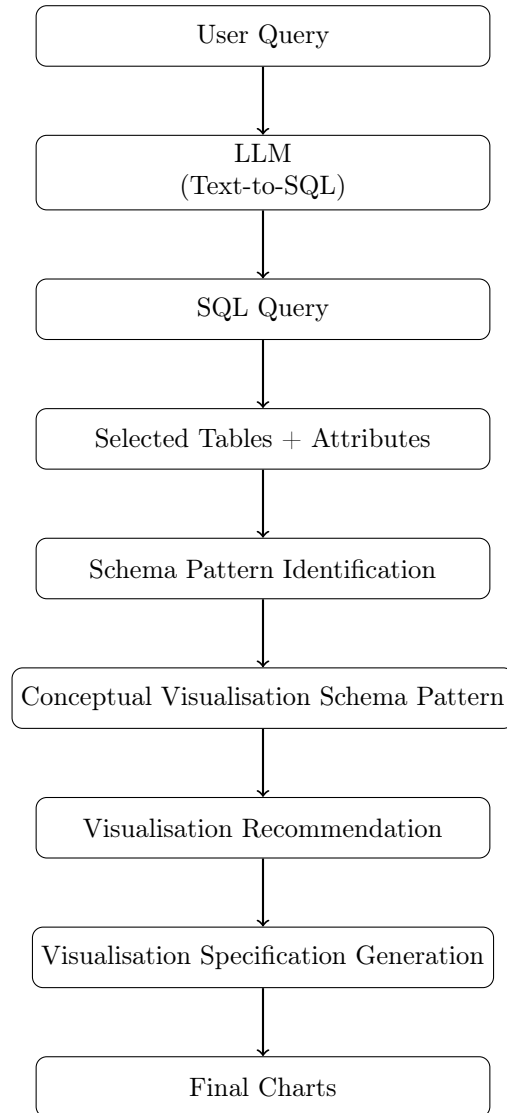


Figure 4.1: Planned project pipeline.

Figure 4.1 illustrates the planned system architecture. The schema pattern identification component has already been implemented and evaluated, while the remaining stages will focus on SQL generation, visualisation recommendation, and automatic visualisation generation.

4.1 Natural Language Query Interpretation

The next stage of the project will investigate how LLMs can transform natural language visualisation requests into executable SQL queries over the Mondial database.

Rather than requiring users to manually specify tables and columns, the system will aim to automatically generate SQL queries from natural language requests. The generated SQL queries will then be analysed to identify the selected tables and attributes, which serve as inputs to the schema pattern identification framework developed in the first stage of the project.

This approach enables the existing schema pattern identification component to be integrated into a complete natural-language-to-visualisation pipeline.

4.2 Visualisation Recommendation

Once the selected tables and attributes have been extracted from the generated SQL query, the schema pattern identification framework developed in the first stage of the project will be applied to determine the corresponding schema pattern.

The project will then investigate whether LLMs can recommend appropriate chart types based on identified schema patterns. Previous work has proposed associations between schema patterns and suitable visualisations; however, it remains unclear whether an LLM can reliably perform this mapping automatically.

Experiments will therefore be conducted to evaluate the extent to which LLM-generated recommendations align with visualisation schema patterns identified in prior research.

The outcome of this stage will be a mapping from conceptual schema patterns to suitable visualisation recommendations, forming the basis for automatic chart selection.

4.3 Data Retrieval and Visualisation Generation

After a suitable visualisation has been selected, the corresponding SQL query will be executed to retrieve the required data from the Mondial database.

The retrieved data and recommended chart type will then be translated into an executable visualisation specification. At this stage, the specific visualisation specification language has not yet been finalised and will be selected following further investigation of available frameworks and their suitability for automatic visualisation generation.

The ultimate objective is to construct an end-to-end pipeline that transforms a natural language query into a visualisation through SQL generation, schema pattern identification, visualisation recommendation, data retrieval, and automatic visualisation specification generation.

4.4 Evaluation

Evaluation will initially focus exclusively on the Mondial database, which serves as the primary experimental environment for this project. Additional databases may be considered only if the proposed approach demonstrates reliable performance on Mondial. The evaluation will consider:

- schema pattern identification accuracy;
- correctness of generated SQL queries;
- visualisation recommendation accuracy;
- correctness of generated visualisation specifications;
- end-to-end task completion success rate.

Where possible, results will be compared against manually constructed ground-truth solutions and previous systems developed using the same conceptual modelling framework.

Period	Planned Activities
May	Completed schema pattern identification experiments using prompt engineering on Mondial test cases derived from previous student reports.
June – Mid-June	Investigate visualisation recommendation based on identified schema patterns and evaluate the effectiveness of LLM-based chart selection.
Mid-June–July	Test and compare the visualization results of different visualization specification languages, and select the best one.
July – Mid-July	Develop a Text-to-SQL module capable of translating natural language visualisation requests into executable SQL queries.
Mid-July–August	Implement automatic data retrieval and visualisation specification generation to construct an end-to-end visualisation pipeline.
August – Mid-August	Conduct end-to-end system evaluation on Mondial test cases and compare results against ground-truth solutions.
Mid-August – September	Complete dissertation writing, system documentation, and final project submission.

Table 4.1: Planned timeline for this project.

4.5 Timeline

Table 4.5 presents the timeline of this project. Please note that during the project’s execution, the tasks may be adjusted and plan may be modified according to the actual situation.

4.6 Definition of Success

The project will be considered successful if the final system can automatically transform natural language visualisation requests into meaningful visualisations through a pipeline consisting of SQL generation, conceptual schema pattern identification, visualisation recommendation, and automatic visualisation specification generation.

In particular, success will be measured by the system’s ability to generate correct SQL queries, identify appropriate conceptual visualisation schema patterns, recommend suitable chart types, and produce executable visualisation specifications without requiring users to manually inspect relational tables.

A further indicator of success would be demonstrating that conceptual-model-based reasoning can support more appropriate visualisation recommendations than approaches that rely solely on relational schema information or tabular data structures.

Bibliography

- [1] Brien PM, Poulouvassilis A. Towards data visualisation based on conceptual modelling and schema transformations.
- [2] Bull D. Using Database Schemas and Conceptual Modelling to Guide Data Visualisation [MSc Final Year Project Report]. London, United Kingdom: Imperial College London; 2023. Accessed: 30 May 2026. Available from: https://www.doc.ic.ac.uk/~pjm/studentprojects/david_bull_msc_final_report.pdf.
- [3] Hannan M. VizER: Data Visualisation Based on Conceptual Modelling [MEng Final Year Project Report]. London, UK: Imperial College London; 2024. Accessed: 30 May 2026. Available from: https://www.doc.ic.ac.uk/~pjm/studentprojects/mohammed_hannan_meng_final_report_2024.pdf.
- [4] May W. The Mondial Database. Institute for Informatics, University of Göttingen; 1999. Available at: <https://www.dbis.informatik.uni-goettingen.de/Mondial/> (Accessed: 2 June 2026).
- [5] Wilkinson L. The Grammar of Graphics. 2nd ed. New York: Springer; 2005.
- [6] Chomsky N. Syntactic structures. Walter de Gruyter; 2002.
- [7] Chomsky N. Aspects of the Theory of Syntax. MIT press; 2014.
- [8] Heer J, Bostock M. Declarative language design for interactive visualization. IEEE transactions on visualization and computer graphics. 2010;16(6):1149-56.
- [9] Wickham H. ggplot2. Wiley interdisciplinary reviews: computational statistics. 2011;3(2):180-5.
- [10] Wickham H. ggplot2: Elegant Graphics for Data Analysis. 2nd ed. Cham, Switzerland: Springer; 2016.
- [11] Satyanarayan A, Moritz D, Wongsuphasawat K, Heer J. Vega-lite: A grammar of interactive graphics. IEEE transactions on visualization and computer graphics. 2016;23(1):341-50.
- [12] IBM Corporation. GPL Reference Guide for IBM SPSS Statistics; 2023. Accessed: 30 May 2026. Available from: https://www.ibm.com/docs/en/SSLVMB_28.0.0/pdf/GPL_Reference_Guide_for_IBM_SPSS_Statistics.pdf.
- [13] Wickham H. A layered grammar of graphics. Journal of computational and graphical statistics. 2010;19(1):3-28.
- [14] Bostock M, Ogievetsky V, Heer J. D³ data-driven documents. IEEE transactions on visualization and computer graphics. 2011;17(12):2301-9.
- [15] Vega Project. Vega: A Visualization Grammar; 2025. Accessed: 2026-05-30. Available from: <https://vega.github.io/vega/>.
- [16] Mackinlay J, Hanrahan P, Stolte C. Show me: Automatic presentation for visual analysis. IEEE transactions on visualization and computer graphics. 2007;13(6):1137-44.
- [17] McBrien P. Topic 14: Entity Relationship Modelling;. Accessed 2026. Lecture Notes, Department of Computing, Imperial College London.

- [18] Ware C. *Information Visualization: Perception for Design*. 2nd ed. Morgan Kaufmann; 2004.
- [19] Brown T, Mann B, Ryder N, Subbiah M, Kaplan JD, Dhariwal P, et al. Language models are few-shot learners. *Advances in neural information processing systems*. 2020;33:1877-901.
- [20] Wei J, Wang X, Schuurmans D, Bosma M, Xia F, Chi E, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*. 2022;35:24824-37.
- [21] Chen M, Tworek J, Jun H, Yuan Q, Pinto HPDO, Kaplan J, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:210703374*. 2021.
- [22] Wang S, Sun X, Li X, Ouyang R, Wu F, Zhang T, et al. Gpt-ner: Named entity recognition via large language models. In: *Findings of the association for computational linguistics: NAACL 2025*; 2025. p. 4257-75.
- [23] Ye Y, Hui B, Yang M, Li B, Huang F, Li Y. Large language models are versatile decomposers: Decomposing evidence and questions for table-based reasoning. In: *Proceedings of the 46th international ACM SIGIR conference on research and development in information retrieval*; 2023. p. 174-84.
- [24] Yu T, Zhang R, Yang K, Yasunaga M, Wang D, Li Z, et al. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task. In: *Proceedings of the 2018 conference on empirical methods in natural language processing*; 2018. p. 3911-21.
- [25] Gao D, Wang H, Li Y, Sun X, Qian Y, Ding B, et al. Text-to-sql empowered by large language models: A benchmark evaluation. *arXiv preprint arXiv:230815363*. 2023.
- [26] Dibia V. LIDA: A tool for automatic generation of grammar-agnostic visualizations and infographics using large language models. In: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*; 2023. p. 113-26.
- [27] Liu P, Yuan W, Fu J, Jiang Z, Hayashi H, Neubig G. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *ACM computing surveys*. 2023;55(9):1-35.
- [28] Kojima T, Gu SS, Reid M, Matsuo Y, Iwasawa Y. Large language models are zero-shot reasoners. *Advances in neural information processing systems*. 2022;35:22199-213.
- [29] Willard BT, Louf R. Efficient guided generation for large language models. *arXiv preprint arXiv:230709702*. 2023.
- [30] Qin B, Hui B, Wang L, Yang M, Li J, Li B, et al. A survey on text-to-sql parsing: Concepts, methods, and future directions. *arXiv preprint arXiv:220813629*. 2022.
- [31] Wang B, Shin R, Liu X, Polozov O, Richardson M. Rat-sql: Relation-aware schema encoding and linking for text-to-sql parsers. In: *Proceedings of the 58th annual meeting of the association for computational linguistics*; 2020. p. 7567-78.
- [32] Cai R, Yuan J, Xu B, Hao Z. Sadga: Structure-aware dual graph aggregation network for text-to-sql. *Advances in Neural Information Processing Systems*. 2021;34:7664-76.
- [33] Cao R, Chen L, Chen Z, Zhao Y, Zhu S, Yu K. LGESQL: Line graph enhanced text-to-SQL model with mixed local and non-local relations. In: *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*; 2021. p. 2541-55.
- [34] Zhong V, Xiong C, Socher R. Seq2sql: Generating structured queries from natural language using reinforcement learning. *arXiv preprint arXiv:170900103*. 2017.
- [35] Narechania A, Srinivasan A, Stasko J. NL4DV: A toolkit for generating analytic specifications for data visualization from natural language queries. *IEEE Transactions on Visualization and Computer Graphics*. 2020;27(2):369-79.

- [36] Maddigan P, Susnjak T. Chat2vis: Generating data visualizations via natural language using chatgpt, codex and gpt-3 large language models. *Ieee Access*. 2023;11:45181-93.
- [37] Wang L, Zhang S, Wang Y, Lim EP, Wang Y. Llm4vis: Explainable visualization recommendation using chatgpt. In: *Proceedings of the 2023 conference on empirical methods in natural language processing: industry track*; 2023. p. 675-92.
- [38] Mackinlay J. Automating the design of graphical presentations of relational information. *Acm Transactions On Graphics (Tog)*. 1986;5(2):110-41.
- [39] Wongsuphasawat K, Moritz D, Anand A, Mackinlay J, Howe B, Heer J. Voyager: Exploratory analysis via faceted browsing of visualization recommendations. *IEEE transactions on visualization and computer graphics*. 2015;22(1):649-58.
- [40] Wongsuphasawat K, Qu Z, Moritz D, Chang R, Ouk F, Anand A, et al. Voyager 2: Augmenting visual analysis with partial view specifications. In: *Proceedings of the 2017 chi conference on human factors in computing systems*; 2017. p. 2648-59.
- [41] Vartak M, Rahman S, Madden S, Parameswaran A, Polyzotis N. Seedb: Efficient data-driven visualization recommendations to support visual analytics. In: *Proceedings of the VLDB Endowment International Conference on Very Large Data Bases*. vol. 8; 2015. p. 2182.
- [42] Hu K, Bakker MA, Li S, Kraska T, Hidalgo C. Vizml: A machine learning approach to visualization recommendation. In: *Proceedings of the 2019 CHI conference on human factors in computing systems*; 2019. p. 1-12.
- [43] Li H, Wang Y, Zhang S, Song Y, Qu H. KG4Vis: A knowledge graph-based approach for visualization recommendation. *IEEE Transactions on Visualization and Computer Graphics*. 2021;28(1):195-205.